

Stratified RMST Boosting for Subgroup Identification

Augmenting the Value Function with Prognostic Strata

Yue Shentu & Natasha

May 29, 2026

small Full simulation results

Contents

1 Introduction

This document defines a stratified version of RMST-based subgroup boosting (“RMST CAV-Boost”). The original method learns a soft subgroup assignment $p_i \in [0, 1]$ (the probability that patient i belongs to the treatment-preferring subgroup) by optimizing a value function through gradient boosting trees. The proposed refinement replaces the original pooled value function with one computed within prognostic strata and aggregated.

We focus on **stratified training**, which replaces the value function used during boosting with the stratified version. The gradient is modified to incorporate stratum-specific RMST differences. This is the version evaluated in the simulations below.

2 Notation

For each subject $i = 1, \dots, N$, we observe:

- $\mathbf{Z}_i$: p -vector of baseline covariates;
- $A_i \in \{0, 1\}$: treatment (1 = active, 0 = control);
- $U_i = \min(T_i, C_i, \tau)$: observed time;
- $\tilde{\delta}_i = \mathbf{1}\{\min(T_i, \tau) \leq C_i\}$: event within τ before censoring.

The restricted mean survival time (RMST) at horizon τ is $\mu(\tau) = \int_0^\tau S(t) dt$, estimated by the area under the Kaplan–Meier curve.

3 Prognostic Stratification

A prognostic score s_i is a univariate summary of $\mathbf{Z}_i$ that predicts survival under the control condition. Because treatment is randomized, we can use all patients (both arms) to estimate s_i as long as treatment is excluded from the predictor set.

We use a cross-fitted elastic net Cox model to estimate s_i , balancing the need to handle $p \approx n/10$ covariates while avoiding overfit within each fold. The implementation:

1. Partition the training data into $F = 5$ random folds.
2. For fold $f = 1, \dots, F$:
 - (a) Fit `cv.glmnet( $\mathbf{Z}$ , Surv( $T, C$ ), family="cox", alpha=0.5)` on the out-of-fold observations.
 - (b) Predict the linear predictor s_i for in-fold observations at `lambda.min`.
3. Collect the out-of-fold predictions $\hat{s}_i$ for all training patients.
4. Define strata by K quantiles of $\hat{s}_i$.

Cross-fitting ensures that the stratum assignment for each patient is independent of its own data, eliminating the training-set overlap bias that would arise from fitting and predicting on the same observations. The elastic net penalty (mixing parameter $\alpha = 0.5$, cross-validated λ) automatically selects among the 52 covariates and shrinks noisy coefficients, which is essential when the Cox

model is proportional-hazards misspecified (e.g., under the quadratic prognostic structure of Zhang scenarios S4–S6).

Stratify s_i into K groups by its quantiles. Let $\mathcal{S}_k$ be the index set of stratum k , with $n_k = |\mathcal{S}_k|$ patients. For $K = 4$, these are quartiles; $K = 6$ and $K = 8$ are used in sensitivity analyses.

4 Stratified Training

In this mode, the value function used during XGBoost training is itself stratified. This changes the gradient that each tree fits, potentially altering the learned p_i .

4.1 Value Function

The stratified value function mirrors the original CAVBoost structure exactly, but applied within each stratum. Recall that the original value function is:

$$V_{\text{orig}} = \left(\sum_i p_i \right) \hat{\Delta}^{(1)} - \left(\sum_i (1 - p_i) \right) \hat{\Delta}^{(2)},$$

where $\hat{\Delta}^{(1)}$ uses p_i -weighted KM (subgroup 1, treatment-preferring) and $\hat{\Delta}^{(2)}$ uses $(1 - p_i)$ -weighted KM (subgroup 2, control-preferring).

The stratified version applies the same logic within each stratum k . Define within-stratum subgroup 1 KM (p -weighted) and subgroup 2 KM ($(1-p)$ -weighted), giving two RMST differences per stratum:

$$\begin{aligned} d_k^{(1)} &= \hat{\mu}_{1,k}^{(1)}(\tau) - \hat{\mu}_{0,k}^{(1)}(\tau), \\ d_k^{(2)} &= \hat{\mu}_{1,k}^{(2)}(\tau) - \hat{\mu}_{0,k}^{(2)}(\tau), \end{aligned}$$

where $\hat{\mu}_{a,k}^{(1)}(\tau)$ is the p -weighted KM RMST within stratum k , arm a , and $\hat{\mu}_{a,k}^{(2)}(\tau)$ is the $(1 - p)$ -weighted equivalent.

The stratified value function is:

$$V_{\text{strat}}(\mathbf{p}) = \sum_{k=1}^K W_k d_k^{(1)} - \sum_{k=1}^K (n_k - W_k) d_k^{(2)}$$

where $W_k = \sum_{i \in \mathcal{S}_k} p_i$ and $n_k = |\mathcal{S}_k|$.

This is the same *difference of deltas* format as the original, but the KM computations are restricted to each stratum. No normalization by N is applied, matching the scale convention of the original.

4.2 Analytic Gradient

The gradient mirrors the original CAVBoost derivation. For patient j in stratum k , taking the derivative of each term in V_{strat} with respect to $\eta_j = \log(p_j/(1 - p_j))$:

$$\begin{aligned} \textbf{Term 1: } \frac{\partial}{\partial \eta_j} [W_k d_k^{(1)}] &= p_j(1 - p_j) d_k^{(1)} + W_k \frac{\partial d_k^{(1)}}{\partial \eta_j} \\ \textbf{Term 2: } -\frac{\partial}{\partial \eta_j} [(n_k - W_k) d_k^{(2)}] &= p_j(1 - p_j) d_k^{(2)} - (n_k - W_k) \frac{\partial d_k^{(2)}}{\partial \eta_j} \end{aligned}$$

The chain rule $\partial d_k^{(g)}/\partial \eta_j = p_j(1 - p_j) \partial d_k^{(g)}/\partial p_j$ gives the full gradient:

$$\frac{\partial V_{\text{strat}}}{\partial \eta_j} = p_j(1 - p_j) \left[d_k^{(1)} + d_k^{(2)} + W_k \frac{\partial d_k^{(1)}}{\partial p_j} - (n_k - W_k) \frac{\partial d_k^{(2)}}{\partial p_j} \right].$$

The XGBoost loss function minimizes, so the gradient passed to the tree learner is $g_j = -\partial V_{\text{strat}}/\partial \eta_j$.

The influence functions. The terms $\partial d_k^{(1)}/\partial p_j$ and $\partial d_k^{(2)}/\partial p_j$ are the derivatives of the within-stratum p -weighted and $(1 - p)$ -weighted RMST differences with respect to patient j 's subgroup probability. They are computed by the same influence-function machinery as the original CAVBoost (the `hazard_inc` function in the R code), evaluated on the events within stratum k only. The per-stratum computation is K times faster than the full-sample version because each stratum has roughly $n_k \approx N/K$ patients.

Block-diagonal structure. A critical property: $\partial d_k^{(g)}/\partial p_j = 0$ for $j \notin \mathcal{S}_k$, because patient j does not contribute to the KM within stratum k . This means the gradient for patient j depends only on patients in the same stratum. The XGBoost regression tree must therefore learn stratum-relevant splits from the feature vector $\mathbf{Z}_j$, relying on the fact that the prognostic score s_i (used to define strata) is a function of $\mathbf{Z}_i$.

Connection to the original. The original CAVBoost gradient (from `rmst_cavboost_clean.R`) is:

$$g_j^{(\text{orig})} = -p_j(1 - p_j) \left[\hat{\Delta}^{(1)} + \left(\sum_i p_i \right) \frac{\partial \hat{\Delta}^{(1)}}{\partial p_j} + \hat{\Delta}^{(2)} - \left(\sum_i (1 - p_i) \right) \frac{\partial \hat{\Delta}^{(2)}}{\partial p_j} \right].$$

The stratified version is identical in structure, but with each term replaced by its stratum-level analogue:

$$\begin{aligned} \hat{\Delta}^{(1)} &\leftrightarrow d_k^{(1)}, & \hat{\Delta}^{(2)} &\leftrightarrow d_k^{(2)}, \\ \sum_i p_i &\leftrightarrow W_k, & \sum_i (1 - p_i) &\leftrightarrow n_k - W_k, \\ \frac{\partial \hat{\Delta}^{(1)}}{\partial p_j} &\leftrightarrow \frac{\partial d_k^{(1)}}{\partial p_j}, & \frac{\partial \hat{\Delta}^{(2)}}{\partial p_j} &\leftrightarrow \frac{\partial d_k^{(2)}}{\partial p_j}. \end{aligned}$$

The implementation is a direct modification of the original: the KM/hazard-increment computation loops over strata instead of the full sample.

4.3 Implementation

The algorithm mirrors the original CAVBoost, but the KM and influence computations loop over strata:

1. Pre-compute cross-fitted prognostic score $\hat{s}_i$ via 5-fold elastic net Cox (`glmnet`, $\alpha = 0.5$) and stratum assignments $\mathcal{S}_k$ by K quantiles. These are fixed throughout boosting.
2. Initialize $\eta_i = 0$ ($p_i = 0.5$).
3. For each boosting round:

- (a) Compute $p_i = 1/(1 + e^{-\eta_i})$.
 - (b) For each stratum k :
 - i. Compute p -weighted KM within $\mathcal{S}_k$, get $d_k^{(1)}$ and influence $\partial d_k^{(1)}/\partial p_j$.
 - ii. Compute $(1 - p)$ -weighted KM within $\mathcal{S}_k$, get $d_k^{(2)}$ and influence $\partial d_k^{(2)}/\partial p_j$.
 - iii. Assemble gradient for patients in $\mathcal{S}_k$: $g_j = -p_j(1 - p_j)[d_k^{(1)} + d_k^{(2)} + W_k \partial d_k^{(1)}/\partial p_j - (n_k - W_k) \partial d_k^{(2)}/\partial p_j]$.
 - (c) Fit a regression tree to $\{g_j, \mathbf{Z}_j\}$.
 - (d) Update $\eta_i \leftarrow \eta_i + \eta_{\text{boost}} \times \text{tree}(\mathbf{Z}_i)$.
4. After training: $\hat{G}_i = \mathbf{1}\{p_i > 0.5\}$.

This is a direct adaptation of the original CAVBoost gradient, replacing the global weighted KM with per-stratum weighted KM. The R functions from `rmst_cavboost_clean.R`—in particular the hazard increment and sorted-data machinery—are reused without modification; only the loop structure changes.

5 Simulation Results

All classification metrics are computed on a held-out test set of 2000 patients (separate from the 500 training patients). Three methods are compared: original CAVBoost, stratified CAVBoost ($K = 4$, cross-fitted elastic net Cox prognostic score), and virtual twin (separate survival random forests via ranger, 200 trees, min node size 10, RMST difference via trapezoidal integration).

5.1 Scenarios

We use the six boundary scenarios from Zhang et al. (2020) with 52 covariates (50 noise, 2 predictive S_1, S_2), $N_{\text{train}} = 500$, $N_{\text{test}} = 2000$, $\tau = 30$, and Setting 2 prognostic strength ($\beta_{Z_{1:4}} = 0.4$). The prognostic score is estimated via 5-fold cross-fitted elastic net Cox (`cv.glmnet`, $\alpha = 0.5$) using all 52 covariates, so the within-stratum gradient computation uses out-of-fold predictions and avoids training-set overlap.

Results from 300 Monte Carlo reps (standard errors in parentheses):

Scenario (boundary)	Orig	Hold-out AUC (SE)			% Correct Dir.
		StratCF	VT	Gain	
S1: Linear ($S_1 > 0$)	0.791 (0.004)	0.974 (0.001)	0.873 (0.002)	+0.184	100%
S2: Diff ($S_1 - S_2 > 0$)	0.793 (0.003)	0.942 (0.001)	0.758 (0.002)	+0.150	100%
S3: U-shaped (S_1)	0.906 (0.003)	0.987 (0.001)	0.867 (0.002)	+0.081	100%
S4: Enclave ($S_1 \times S_2$)	0.893 (0.002)	0.813 (0.003)	0.960 (0.001)	−0.080	70%
S5: S-shaped (S_1 , disjoint)	0.952 (0.002)	0.946 (0.003)	0.786 (0.002)	−0.006	72%
S6: Cross ($S_1 \times S_2$)	0.928 (0.004)	0.944 (0.005)	0.780 (0.004)	+0.017	80%
<i>Average</i>	0.877	0.934	0.838	+0.058	87%

The key findings are:

- **Stratified CAVBoost dominates on additive and monotone boundaries** (S1–S3, gains of 8–18 AUC points). The cross-fitted elastic net prognostic score absorbs the strong prognostic effects of Z_1 – Z_4 and aligns with the smooth treatment-effect surface. Within homogeneous strata, the gradient sees a nearly clean signal, producing near-perfect AUC values on S1 (0.974) and S3 (0.987).
- **Stratification is detrimental on disjoint interaction boundaries** (S4 Enclave: StratCF 0.813 vs. Orig 0.893). The Enclave boundary is a disconnected rectangular region in $S_1 \times S_2$ space. A linear Cox PH prognostic score cannot capture this structure, so the strata misalign with the true subgroup boundary and the gradient becomes unfocused. The virtual twin (non-parametric ranger) performs best here (0.960), as the survival forest naturally finds the interaction.
- **On complex but connected boundaries** (S5 S-shaped, S6 Cross), the stratified method is approximately neutral (S5: -0.006 , S6: $+0.017$). The prognostic score partly captures the structure but not cleanly enough to produce a large gain.
- **The stratified method never collapses.** Across all six scenarios, the worst-case loss relative to the original is 8 AUC points (S4). In no scenario does it fall below 0.81, and the proportion of correctly directed predictions ($\text{AUC} > 0.5$) is 70–100% across scenarios.
- **Original CAVBoost is robust** (0.79–0.95 AUC), making it a reliable all-purpose baseline. It wins scenario S5 outright and ties on S6, demonstrating that the unstratified value function is already competitive on complex boundaries.

5.2 Cross-Validation Across Independent Runs

To verify reproducibility, the simulation was run independently on two machines with different random seeds:

- Agent server: 200 reps, seed base 42
- HP Omen: 300 reps, seed base 12345

Results for Scenario S1 were nearly identical (Orig: 0.788 vs. 0.791, StratCF: 0.974 vs. 0.974, VT: 0.870 vs. 0.873), confirming that the observed AUC values are not artifacts of a particular seed or hardware configuration. The combined dataset provides 500 independent reps per scenario for downstream analysis.

5.3 Practical Implications

The results suggest a practical rule: **use stratified CAVBoost as the default, but check for sensitivity on interaction-dominated boundaries.** When the treatment-effect boundary is smooth relative to prognosis (the common case in oncology trials driven by a few strong biomarkers), the stratified gradient extracts substantially more information from the data. When the boundary involves complex interactions that a linear prognostic score cannot capture, fall back to the original method or a survival forest-based virtual twin.

The cross-fitted elastic net prognostic score is essential to this approach. Using fewer than the full set of covariates silently changes the score and can produce misleading comparisons, as we discovered during development. The implementation includes a runtime assertion (`stopifnot`) guarding the 52-covariate requirement.

6 Discussion

The stratified RMST boosting method presented here offers a simple, computationally efficient modification of the original CAVBoost algorithm. By computing the value function within prognostic strata, the gradient is shielded from prognostic confounding, leading to substantial improvements in subgroup identification accuracy on smooth or monotone treatment-effect boundaries.

The method requires only:

1. A one-time computation of cross-fitted prognostic strata (elastic net Cox, < 1 second for $n = 500$);
2. Replacing the global weighted KM loop with a per-stratum loop (K smaller loops instead of one large loop, actually faster per iteration because each stratum is N/K patients).

The key limitation is the linearity of the Cox PH prognostic score. When the true subgroup boundary involves interactions that a linear score cannot represent, stratification may misalign the gradient. Extending the prognostic model to capture non-linear structure—via random survival forests, gradient-boosted survival trees, or foundation model embeddings—is a natural direction for future work.